

EC-333 Microprocessor and Interfacing Techniques

Experiment # 6 & 7

Introduction to MPLAB X IDE

1. Objective/s:

- Creating a New Project
- Overview of IDE
- Configuration Bits
- Program Structure
- Debugging Mode
- Programming the Device
- Lab Task

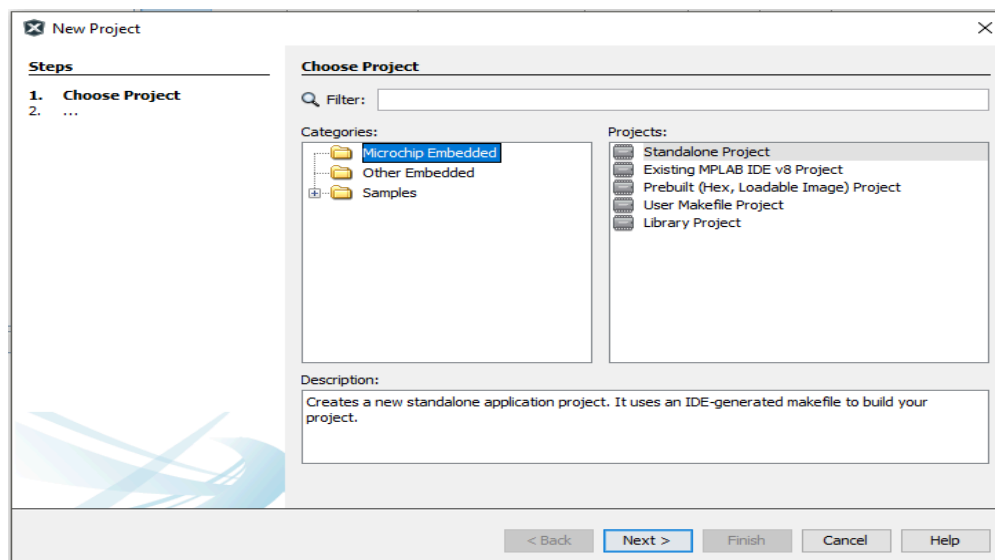
2. Opening a New Project in MPLAB X

This section describes how to create a project in MPLAB X IDE using the MPLAB XC8 C Compiler.

The process is explained in the following steps:

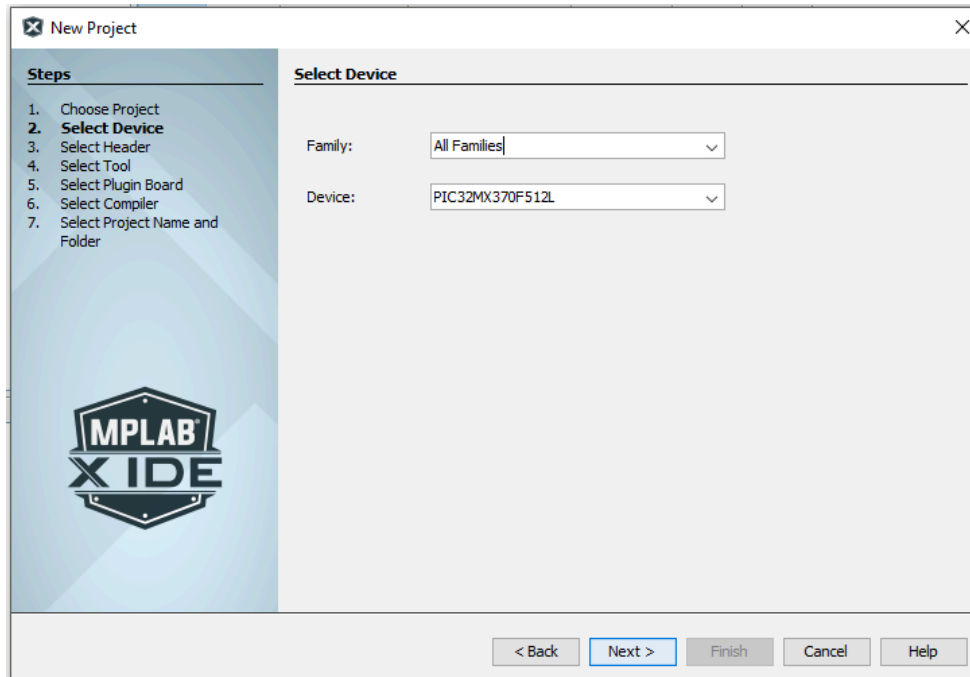
1. Sets the project type.

From MPLAB X IDE, choose *File>New Project....* In the window that opens, select the “Microchip Embedded” category, and a “Standalone Project” from the Projects field.



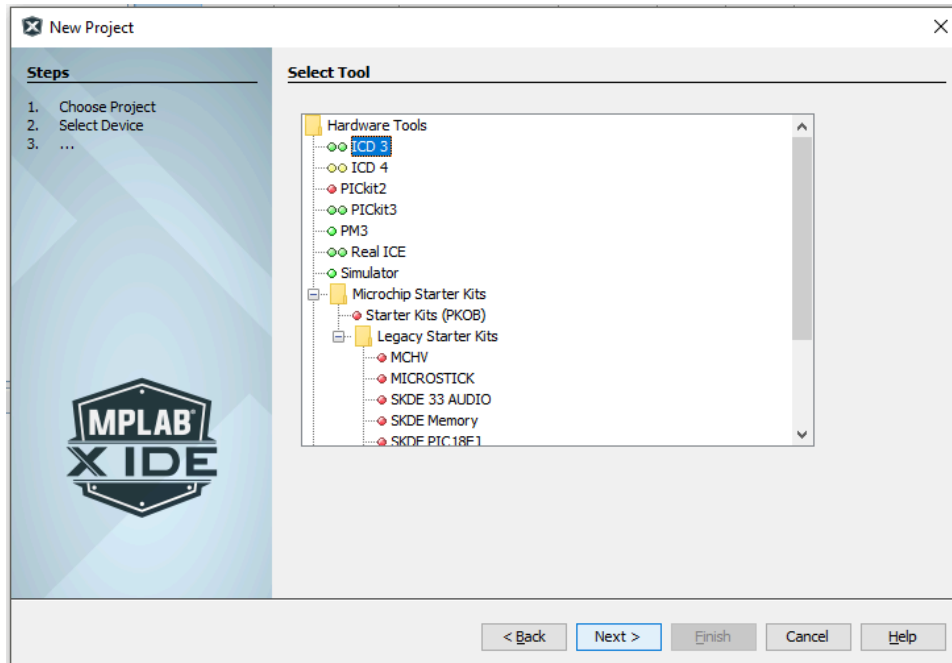
2. Selects the target device.

This selection must exactly match the device on your hardware. (If you are using the simulator without hardware, you can choose any device.). MPLAB XC8 can compile for any device in the 32-bit-microcontroller families. In Figure, a PIC32MX370f512L has been selected



3. Selects the tool to run the project code.

If you have a debugger and wish to use it with your hardware, select it from the list;

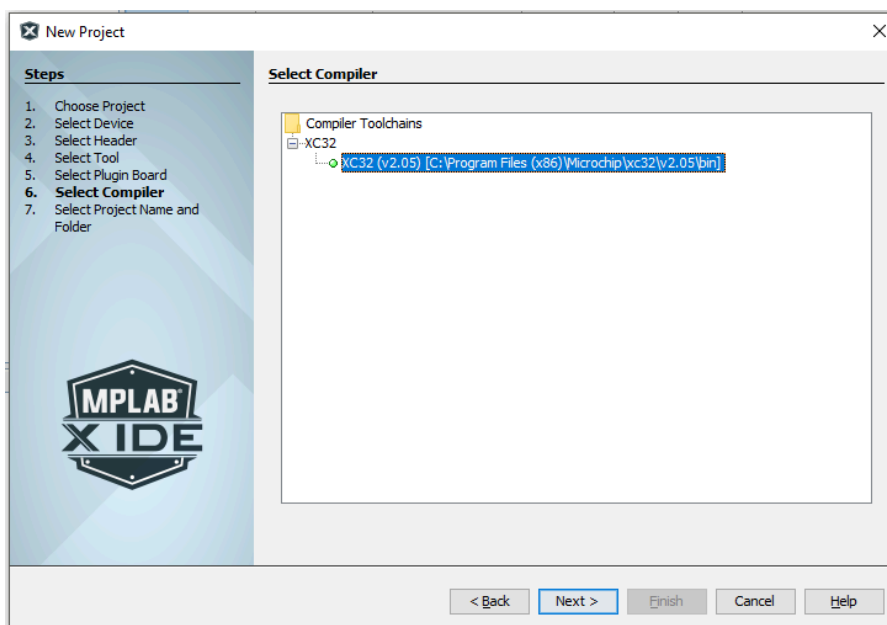


4. This step is only applicable to some debugger tool selections.

Unless you must use a specific plugin board, select None.

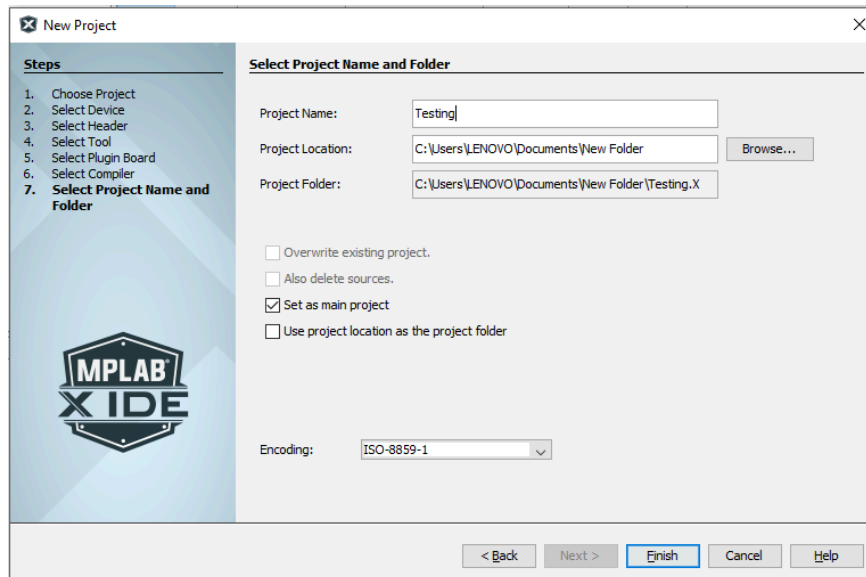
5. Selects the tool to compile the source code.

As shown in the Select Compiler window in Figure, select a suitable compiler for your board.
We will select XC32 compiler



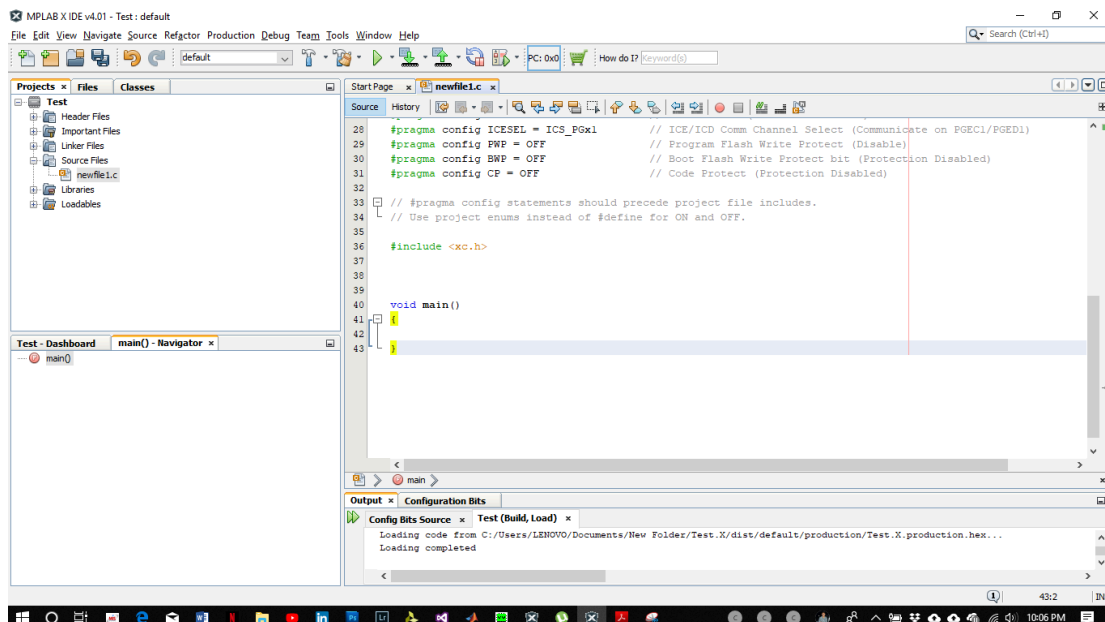
6. Specifies the project name and path.

Type a name for the project in the Project Name field.



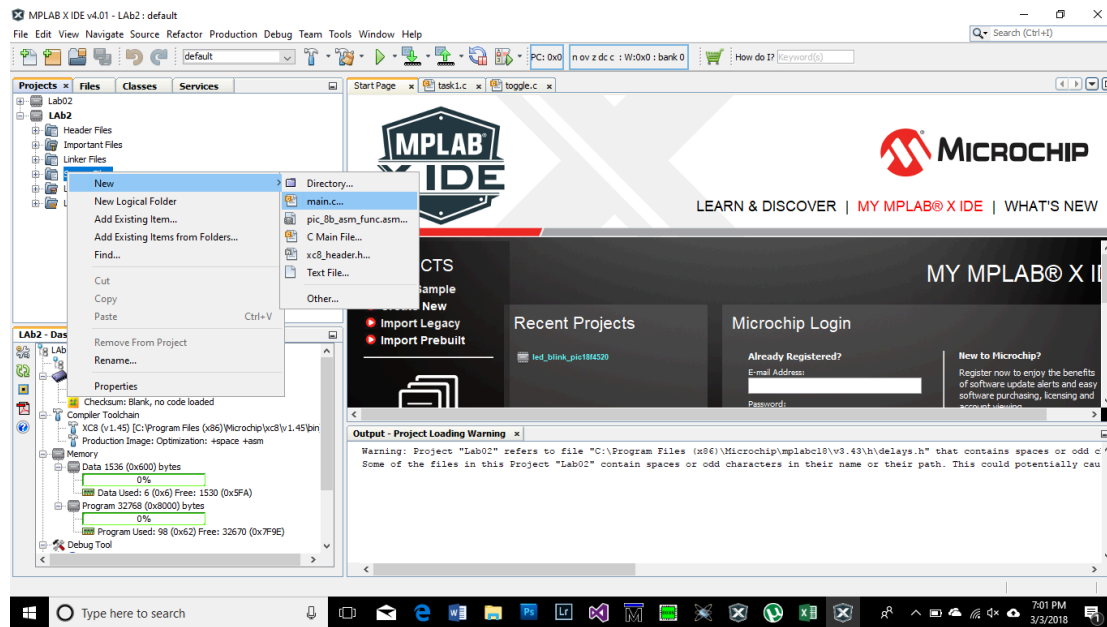
7. Completes creation of the project.

Click **Finish**, and the project is created. An icon representing the project appears in the Projects window¹. The Projects window is shown at the top left in the figure. Below the Projects window, the Dashboard provides more detailed project information.



8. Adding new file to workspace.

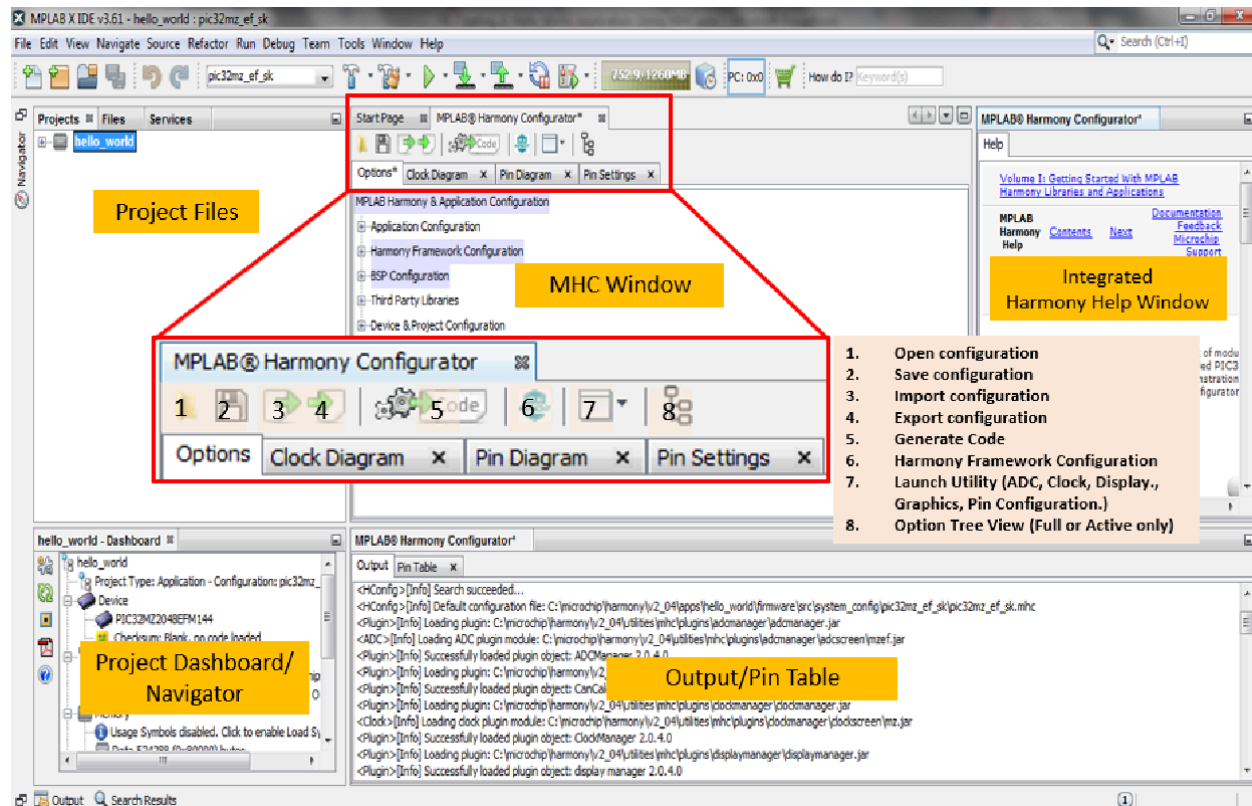
Upon successful creation of project, add new file to the project by right clicking on the source file and adding new file.



9. Building the solution.

Write the code in the workspace loaded and build the code by clicking on the build solution icon

3. Overview of IDE



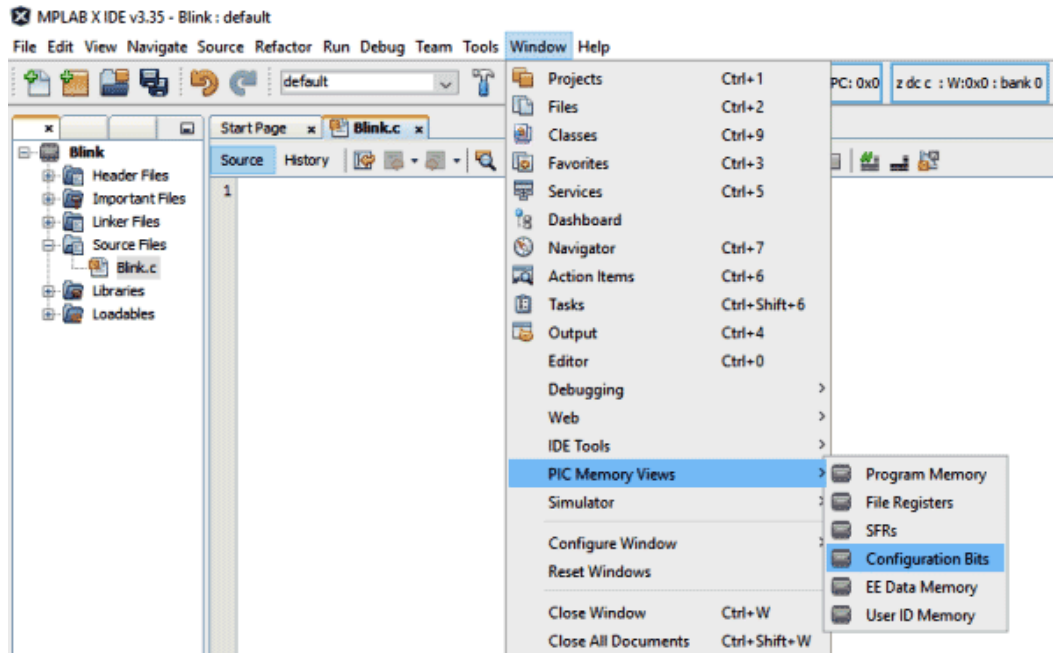
4. Setting Configuration Bits

The PIC devices have several locations, which contain the Configuration bits or fuses. These bits specify fundamental device operation, such as the oscillator mode, watchdog timer, programming mode and code protection. These bits must be set correctly, in order to run the code otherwise we have non-running device. So, it is very important to know about these configuration Registers before even we start with our Blink Program.

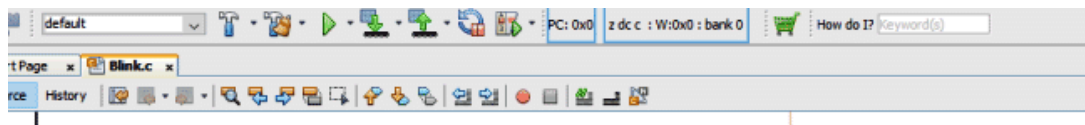
In order to use the Configuration registers we have to read through the Datasheet (refer to Chap. 28. Special Features) and understand what the different types of Configuration bits are available and their functions. These bits can be set or reset based on our programming requirements using a configuration pragma.

Microchip has made this tiring process a lot easier by using graphical representations of the different types of Configuration bits. So now in order to set them we simply have to follow the steps below.

Click on Window -> PIC Memory View -> Configuration Bits. As shown below.



This should open the Configuration Bits window in the bottom of our IDE as shown below. This is the place where we can set each of the configuration bits according to our needs. Each of the bits and its purpose will be explained later as we progress through the steps.



Address	Name	Value	Field	Option	Category	Setting
2007	CONFIG	FFFF	POSC	EXTRC	Oscillator Selection bits	RC oscillator
			WDTE	ON	Watchdog Timer Enable bit	WDT enabled
			PWRT	OFF	Power-up Timer Enable bit	PWRT disabled
			BOREN	ON	Brown-out Reset Enable bit	BOR enabled
			LVP	ON	Low-Voltage (Single-Supply) In-Circuit Serial Programming Enable bit	RB3/PGM pin has PGM fu
			CPD	OFF	Data EEPROM Memory Code Protection bit	Data EEPROM code prote
			WRT	OFF	Flash Program Memory Write Enable bits	Write protection off;
			CP	OFF	Flash Program Memory Code Protection bit	Code protection off

4.1. Oscillator

The PIC32 can be operated in four different oscillator modes. These four modes can be selected by programming POSCMOD configuration bits:

XT Crystal/Resonator

HS High-Speed Crystal/Resonator

EC

Osc Type	Crystal Freq.	Cap. Range C1	Cap. Range C2
LP	32 kHz	33 pF	33 pF
	200 kHz	15 pF	15 pF
XT	200 kHz	47-68 pF	47-68 pF
	1 MHz	15 pF	15 pF
	4 MHz	15 pF	15 pF
HS	4 MHz	15 pF	15 pF
	8 MHz	15-33 pF	15-33 pF
	20 MHz	15-33 pF	15-33 pF

4.2. Watchdog Timer

The Watchdog Timer is a free running, on-chip RC oscillator which does not require any external components. This RC oscillator is separate from the RC oscillator of the OSC1/CLKI pin. That means that the WDT will run even if the clock on the OSC1/CLKI and OSC2/CLKO pins of the device has been stopped. During normal operation, a WDT time-out generates a device Reset (Watchdog Timer Reset). The TO bit in the Status register will be cleared upon a Watchdog Timer time-out. If the timer is not cleared in our software coding then the whole MCU will reset upon every WDT timer overflow. The WDT can be permanently disabled by clearing configuration bit.

We are not using WDT in our program so let us clear it, by selecting OFF from the dropdown box.

Now click on Generate Source Code to Output, our code will be generated now just copy it along with header file and paste in on your .C File, as shown below

```

// CONFIG
#pragma config FOSC = HS      // Oscillator Selection bits (HS oscillator)
#pragma config WDTE = OFF     // Watchdog Timer Enable bit (WDT disabled)
#pragma config FWRT = OFF     // Power-up Timer Enable bit (PWRT disabled)
#pragma config BOREN = ON     // Brown-out Reset Enable bit (BOR enabled)
#pragma config LVP = OFF      // Low-Voltage (Single-Supply) In-Circuit Serial Programming Enable bit (RB3 is digital I/O, HV on MCLR must be used for programming)
#pragma config CPD = OFF      // Data EEPROM Memory Code Protection bit (Data EEPROM code protection off)
#pragma config WRT = OFF      // Flash Program Memory Write Enable bits (Write protection off; all program memory may be written to by EECON control)
#pragma config CP = OFF       // Flash Program Memory Code Protection bit (Code protection off)

// #pragma config statements should precede project file includes.
// Use project enums instead of #define for ON and OFF.

#include <xc.h>
  
```


5. Program Structure

```
// DEVCFG0
#pragma config DEBUG = OFF           // Background Debugger Enable (Debugger is Disabled)
#pragma config JTAGEN = ON           // JTAG Enable (JTAG Port Enabled)
#pragma config ICESEL = ICS_PGx1    // ICE/ICD Comm Channel Select (Communicate on PGEC1/PGED1)
#pragma config PWP = OFF             // Program Flash Write Protect (Disable)
#pragma config BWP = OFF             // Boot Flash Write Protect bit (Protection Disabled)
#pragma config CP = OFF              // Code Protect (Protection Disabled)

#include <xc.h>                       //Libraries

void main()
{
    //Setting Direction of I/O Pins
    //Declaration of Variables
    //Configuration of Registers of Devices

    while(1)
    {
        //Infinite Loop
        //Functionality of Program
    }
}
```

6. Sample Code

Write a sample code with empty main block and try to build the solution, there must be no errors, then try to add some instructions like making PORTA an input and PORTB an output. Build again and see if there are any error. This is also the right approach to write a code. After every portion of task (writing instructions) code must be build in order to validate the legitimacy of code

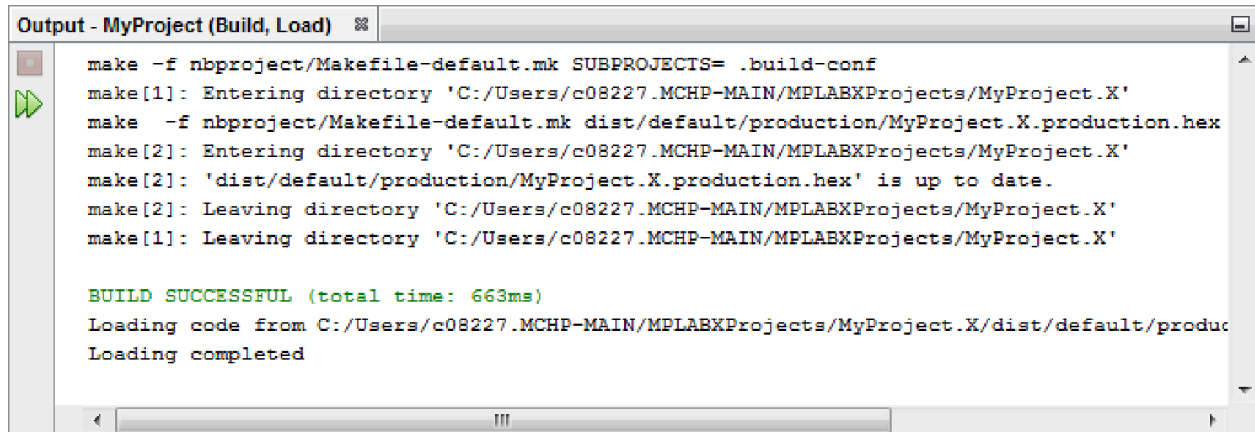
7. Running and Debugging

For MPLAB X IDE, it is not necessary to build the project first and then run or debug. Building is part of the run and debug processes. For initial development or major changes, however, you may want to make sure that the project builds before attempting to run or debug.

7.1. To build a project:

- In the Projects window, right click on the project name and select “Build”. You may also select “Clean and Build” to remove intermediary files before building.
- Click on the “Build Project” or “Clean and Build Project” toolbar icon.

Build progress will be visible in the Output window, located in the Task Pane.



```

Output - MyProject (Build, Load)
make -f nbproject/Makefile-default.mk SUBPROJECTS= .build-conf
make[1]: Entering directory 'C:/Users/c08227.MCHP-MAIN/MPLABXProjects/MyProject.X'
make -f nbproject/Makefile-default.mk dist/default/production/MyProject.X.production.hex
make[2]: Entering directory 'C:/Users/c08227.MCHP-MAIN/MPLABXProjects/MyProject.X'
make[2]: 'dist/default/production/MyProject.X.production.hex' is up to date.
make[2]: Leaving directory 'C:/Users/c08227.MCHP-MAIN/MPLABXProjects/MyProject.X'
make[1]: Leaving directory 'C:/Users/c08227.MCHP-MAIN/MPLABXProjects/MyProject.X'

BUILD SUCCESSFUL (total time: 663ms)
Loading code from C:/Users/c08227.MCHP-MAIN/MPLABXProjects/MyProject.X/dist/default/production/MyProject.X.production.hex
Loading completed
  
```

7.2. Run Code

After the code builds successfully, you can attempt to run the application. Click on the “Make and Program Device Project” icon (or select *Run>Run Project*) to run your program.



Make and Program Device Project Icon

Run progress will be visible in the Output window as well.

Use the Hold in Reset button to toggle between device Reset and running.



Hold in Reset Icon

You can add a “Run Project” icon to the toolbar if you wish



Run Icon

7.3. Debug Run Code

For this tutorial, we’ve assumed that the code used has been tested and runs successfully. However, your own code may need to be debugged as you develop your application. To Debug Run the tutorial code, click on the “Debug Project” icon (or select *Debug>Debug Project* or *Debug>Step Into*) to begin a debug session.



Debug Run Icon

Debug Run progress will be visible in the Output window.

To halt your application code:

Click on the “Pause” icon (or select *Debug>Pause*) to halt your program execution.

To run your code again:

Click on the “Continue” icon (or select *Debug>Continue*) to start your program execution again.

To end execution of your code:

Click on the “Finish Debugger Session” icon (or select *Debug>Finish Debugger Session*) to end your program execution.

Control Program Execution with Breakpoints

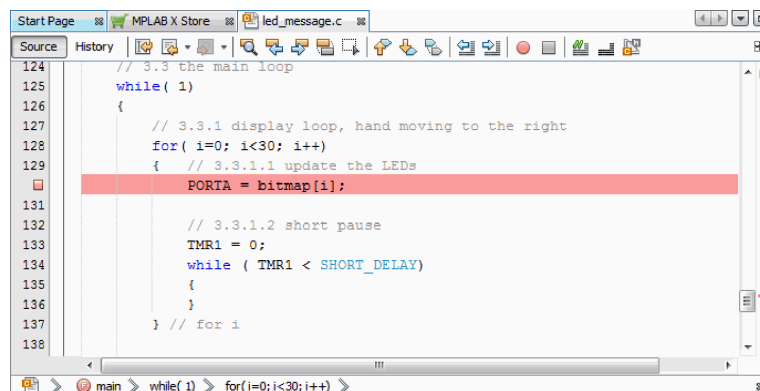
When debugging code, it can be useful to suspend execution at a specific location in code so that variable values can be examined. To do this, use breakpoints.

Set a breakpoint on the following line of code:

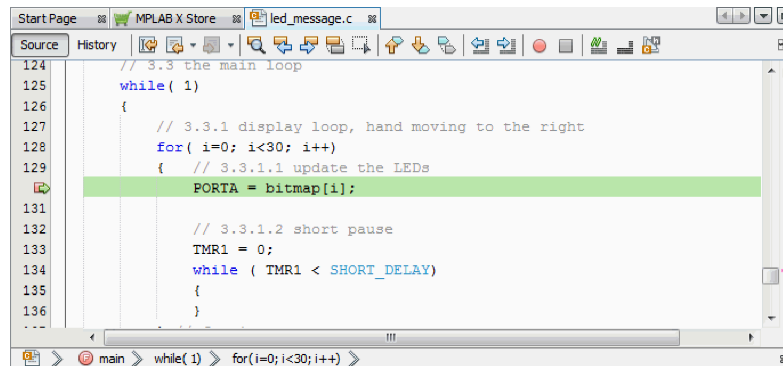
```
PORTA = bitmap[i];
```

To set a breakpoint on a line, do one of the following:

- Click the left margin of the line in the Source Editor
- Press Ctrl-F8



Debug Run the tutorial program again. The program will halt at the breakpoint. Hover over the `bitmap[ ]` variable to see its values.



7.4. Step Through Code

Use one of the stepping functions on the Debug menu and Debug toolbar to move through code either from the beginning of code or after a breakpoint halt. Examine changes in variable values (see next section) or determine if the program flow is correct.

There are several ways to step through code:

- Step Over – Executes one source line of a program. If the line is a function call, it executes the entire function and stops.
- Step Into – Executes one source line of a program. If the line is a function call, it executes the program up to the function's first statement and stops.
- Step Out – Executes one source line of a program. If the line is a function call, it executes the functions and returns control to the caller.
- Run to Cursor – Runs the current project to the cursor's location in the file and stops program execution.

7.5. Watch Symbol Values Change

Watch the values of symbols that you select change in the Watches window. Determining if these values are as expected during program execution will help you to debug your code.

To create a new watch:

1. Select *Debug>New Watch*. The New Watch dialog will open (see below).
2. Enter a Watch expression, in this case `PORTA`, and then click **OK**. The Watches window will now appear on the desktop with the symbol.

OR

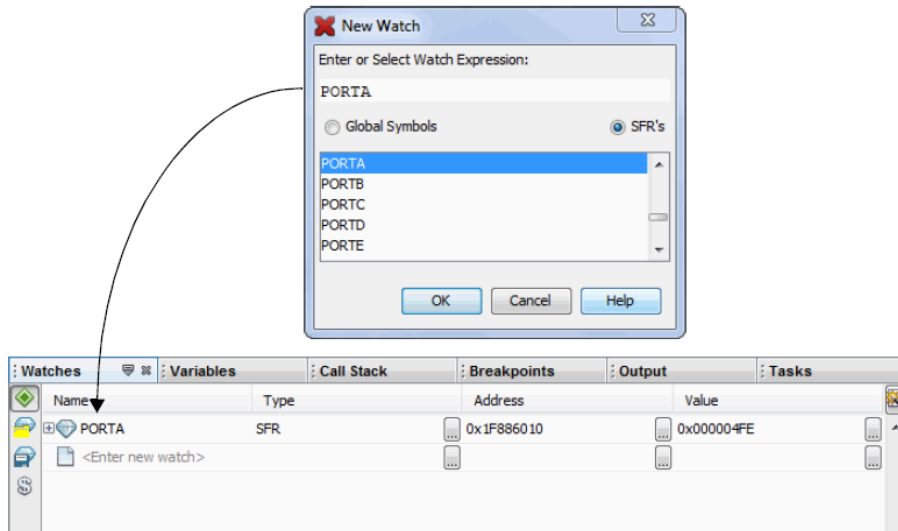
1. Right click on `PORTA` in code and select "New Watch".

2. The Watches window will open with PORTA in it.

OR

1. Select *Window>Debugging>Watches*.

2. Drag and drop PORTA from the editor window into the Watches window.



7.6. View Device Memory (including Configuration Bits)

MPLAB X IDE has flexible, abstracted memory windows that provide a customizable view of differing types of device memory. From a Memory window, select the type of memory and memory format in drop-down boxes.

Begin by viewing Flash memory:

1. Select *Window>PIC Memory Views>Execution Memory*.

2. The Execution Memory window will open showing the last halt location.

Execution Memory		Watches		Output	
	Line	Address	Opcode	Label	DisAssy
	8265	9D00...	34038030		ORI V1, ZERO, -32720
	8266	9D00...	AC430600		SW V1, 1536(V0)
	8267	9D00...	AFC00000		SW ZERO, 0(S8)
	8268	9D00...	0B40005E		J 0x9D000178
	8269	9D00...	00000000		NOP
	8270	9D00...	3C02A000		LUI V0, -24576
	8271	9D00...	24430200		ADDIU V1, V0, 512
	8272	9D00...	8FC20000		LW V0, 0(S8)
	8273	9D00...	00621021		ADDU V0, V1, V0
	8274	9D00...	80420000		LB V0, 0(V0)
	8275	9D00...	00401821		ADDU V1, V0, ZERO

7.7. Program a Device

After your code is debugged, you can program it onto a target device.

In order to program the device, there are two ways:

- Click **Run**: The project is built (if necessary) and device is programmed. The program will immediately begin execution on completion of programming.
- Click **Make and Program Device**: The project is built (if necessary) and device is programmed. The program will NOT immediately begin execution on completion of programming. You will need to disconnect the hardware tool from the target board before the application will run.

Other programming-related functions are:

- **Hold in Reset**: Toggle the device between Reset and Run.
- **Read Device Memory**: Transfer what is in target memory to MPLAB X IDE.

Programming-related icons are:



Run Icon



Hold In Reset Icon



Make and Program Device Icon



Read Device Memory Icon

8. Tasks

For the following tasks, select **simulator** as a hardware tool while creating the project.

1. Write a program for a mathematical expression in which you define two variables, add them together, and store the result into third variable. Build the program and then debug the program to execute and analyze the code line by line by using **step into** tool. Use **variable** tool to analyze the values of each variable.
2. Write a program to find the square of a predefined number using functions. Debug the program to analyze the flow of the program. Use **step into** and **step over** tool to analyze how the debugger shows the execution of instructions of function and how it skips that respectively. Use **variable** tool to analyze the values of each variable.

For the following tasks, select **BASYS** as a hardware tool while creating the project.

3. Write a “blink led” program for BASYS MX3. Using any desired slide switch, any desired led should turn on/off by turning the switch on/off. Debug the program and flash the board using following the steps mentioned above.
4. If we debug the above program while board still connected to the system and BASYS being selected as a hardware tool, we can debug the program on the board instead of a simulator. Debug the above program and analyze the values of SWITCH and LED in the **watch** tool by executing the instructions line by line using the step into tool and by toggling the state of SWITCH.